

Configuration from the environment

Read settings from the computer the app runs on, and turn text into real types.

The big idea

The same code runs on your laptop and on the real server (**production**). Each place needs different **settings**: passwords, keys, **DEBUG** on or off.

The code is a recipe; **environment variables** are the oven dial in each kitchen. The catch: they are **always text**, so **DEBUG=False** arrives as the string **"False"**.

What your code receives

The tests set a variable for a moment, then pass its **name** to your function.

```
# 1. The test sets a variable (always text!)
os.environ["APP_PORT"] = " 8000 "
# 2. It calls your function with the variable's NAME
env_int("APP_PORT", 80)                # must return 8000, a number

# More calls from the tests, and what they expect:
env_str("APP_NAME", "fallback")        # not set -> "fallback"
env_str("APP_NAME")                    # not set -> ImproperlyConfigured
env_bool("APP_DEBUG", default=True)    # " OFF " -> False
env_bool("APP_DEBUG")                  # "maybe" -> ImproperlyConfigured
env_list("APP_HOSTS")                  # " localhost, 127.0.0.1 ,,example.com, "
                                      # -> ["localhost", "127.0.0.1", "example.com"]
```

New words

environment variable A `NAME=value` setting that lives outside your code.

production The real server real users use. Mistakes here hurt.

`os.environ` Python's view of all environment variables: a dict of strings.

truthy What `if` treats as true: every non-empty string, even `"False"`.

default The value to use when the variable isn't set.

raise Stop the function right away with an error.

Tool: `os.environ.get(name)`

A **method** on `os.environ`: gives the value as a **string**, or `None` if it isn't set.

```
import os

os.environ["MY_COLOUR"] = " Blue "      # set one, just to try it
print(repr(os.environ.get("MY_COLOUR")))
print(os.environ.get("NOT_SET_AT_ALL"))
```

OUTPUT

```
' Blue '
```



```
None
```

Why "False" is truthy

Only the empty string is false, so `bool(...)` can't read settings.

```
print(bool("False"))    # the classic bug
print(bool("0"))
print(bool(""))
```

OUTPUT

True

True

False

Tools: is None, .strip() and in {...}

Check "missing or blank" first, then compare the cleaned word against a **set**.

```
value = "  "  
print(value is None or not value.strip())    # blank: use the default  
  
SIZES = {"small", "medium", "large"}  
answer = " Medium "  
print(answer.strip().lower() in SIZES)
```

OUTPUT

True

True

Tool: raise ImproperlyConfigured(...)

Stops the function with a clear message. Put the variable's name in it with an f-string.

```
from django.core.exceptions import ImproperlyConfigured

name = "MY_COLOUR"
raise ImproperlyConfigured(f"Please set {name}.")
```

OUTPUT

```
django.core.exceptions.ImproperlyConfigured: Please set MY_COLOUR.
```

Tool: `int(text)` with `try / except`

`int(" 42 ")` works, `int("forty")` raises `ValueError`, and `except` catches it.

```
for text in ["42", "forty"]:
    try:
        print(int(text) + 1)
    except ValueError:
        print(f"{text!r} is not a number")
```

OUTPUT

43

'forty' is not a number

return vs raise

return the error

- Hands the error back as a **normal value**
- The function carries on as if all is fine
- The app runs with a broken setting
- The tests see no error: tests 2 and 5 fail

raise the error

- **Stops** the function right away
- Nothing after it runs
- The app refuses to start, with a clear message
- What the tests expect

Pattern: split, clean, keep the good ones

The same loop `env_list` needs, on different data:

```
raw = " red , ,green,, blue "  
colours = []  
for piece in raw.split(","):  
    piece = piece.strip()  
    if piece:                                # "" is falsy, so empty pieces are skipped  
        colours.append(piece)  
print(colours)
```

OUTPUT

```
['red', 'green', 'blue']
```

Your task, step by step

- 1 In every helper: read with `os.environ.get(name)`; `None` or blank means **missing**
- 2 `env_str`: return the stripped value, or `default` → **test 1**
- 3 `env_str`: missing and no default? `raise ImproperlyConfigured` naming it → **test 2**
- 4 `env_bool`: lower-case, check the true set and the false set, else raise → **tests 3-5**
- 5 `env_int`: `int(value)` inside `try`; raise on `ValueError` → **test 6**
- 6 `env_list`: missing gives `default` or `[]`; else split, strip, skip empties → **tests 7-8**

Common mistakes

- `bool(value)` makes `"False"` true; `value == "True"` misses `"true"` and `"1"`
- Calling `.strip()` before checking `is None`: `AttributeError`
- Forgetting that `" "` (only spaces) counts as missing
- `return ImproperlyConfigured(...)` instead of `raise`
- `env_list` returning `None` instead of `[]`; `env_int` returning `"8000"`

Where you'll use this

You deploy to production and set `DJANGO_DEBUG=tur` by mistake. With `bool(...)` the site quietly runs in debug mode and shows error details to strangers. With `env_bool`, it refuses to start and tells you why.

- Real projects read `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS` and database settings this way
- Open `.env.example` and `config/settings.py` in this project to see it
- Libraries like `django-environ` do the same job as your four helpers

Check yourself

1. Why keep passwords and `SECRET_KEY` out of `settings.py`?
2. `DJANGO_DEBUG=False`: what does `os.environ.get("DJANGO_DEBUG")` give, and is it truthy?
3. Why check `value is None` before calling `.strip()`?
4. What is the difference between `return` and `raise` for an error?

Keep learning

[Python Env Vars – How to Get an Environment Variable in Python](#)

freeCodeCamp · article · beginner. Start here: `os.environ` works like a dict, and `os.environ.get` lets you supply a default when a variable is missing.

[The Twelve-Factor App: III. Config](#)

The Twelve-Factor App · article · beginner. A one-page explanation of why settings and secrets belong in the environment, not in your code.

[os.environ](#)

Python docs · docs · intermediate. Official reference; note that every value is a string, which is why you write `env_bool`, `env_int` and `env_list`.

[Django settings](#)

Django docs · docs · intermediate. How Django loads `settings.py`, where your env helpers end up being used.

[Deployment checklist](#)

Django docs · docs · intermediate. Lists the settings (`DEBUG`, `SECRET_KEY`, `ALLOWED_HOSTS`) that must come from the environment in production.